bev-decider in the multi-agent: ideas and how much decode each one removes

FieldMind, Embedded Systems Workshop Lab 2, Project 4 · branch `bev-decider-3` · 12 October 2026 · estimates from the measured v3 board run, not from new model runs

The idea in one paragraph

bev-decider-0.4B is a **decision model, not a chat model**. It reads a text state and typed questions (*choice* of named options, *yes/no*, or an ordinal *score*) and returns calibrated probabilities in **one prefill pass, with zero decode tokens**. Decode is where our time goes: on the board, generating the answer takes **69%** of every Llama diagnosis call. So bev saves decode only where it **replaces** a generated answer or **cancels** a call that would not have changed anything. Adding bev next to an existing call adds cost, it does not remove any.

19,785
decode tokens generated by the v3 multi-agent on the board (12 episodes, 2,100 ticks): Llama 17,284 + Gemma verifier 2,501

64.3 ms
per decoded token, Llama 3.2 3B on the NPU (measured). Reading the prompt costs 1.27 ms per token: **a generated token costs ~50× a read one**

−68.6%
upper bound on decode removed if ideas 2, 4 and 6 all worked perfectly (19,785 → 6,220 tokens). Real savings depend on bev's accuracy, unmeasured

Where bev can enter each task

"Start" means before a generated call (decide whether or what to ask). "Middle" means taking part of the answer away from the generating model. "End" means after the call (a fallback when it fails). "Replace" means bev does the whole job instead of a generating model.

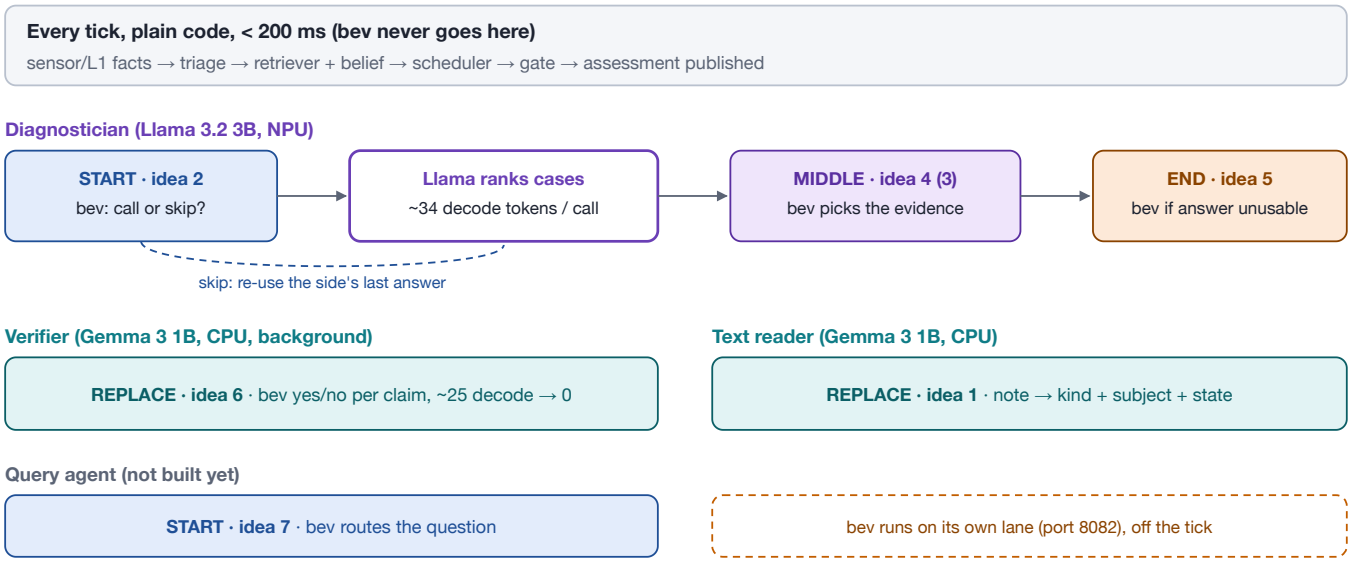


Figure 1. The seven insertion points. Idea 3 (narrow the candidates) sits with idea 4 in the middle but turned out to save no decode, see page 2.

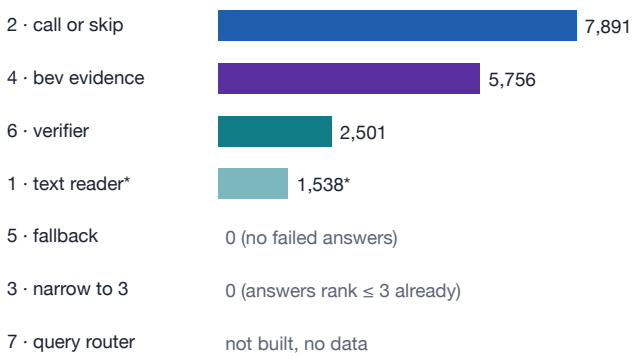
The answer first: what each idea can remove

Measured on the v3 board run (Llama 3.2 3B on the NPU, Gemma 3 1B on the CPU, 12 episodes, lockstep). "Max decode removed" assumes bev does its part perfectly, so every figure is a **ceiling, not a forecast**. Times are model call times measured on the board, not energy (energy was not measured).

#	idea	where	decode today	max removed	% of all decode	time removed	bev's extra work	verdict
2	Call or skip the diagnostician	start	17,284	7,891	**39.9%**	733 s NPU 45.7% of Llama call time	1 choice / eligible call (236)	biggest riskiest
4	bev picks the supporting facts	middle	17,284	5,756	**29.1%**	369 s NPU decode only	1 choice / ranked case (~1,222)	second
6	Replace the verifier	replace	2,501	2,501	**12.6%**	189 s CPU whole call, background lane	1 yes/no per claim (~300)	measure first
1	Replace the text reader	replace	0 in v3*	1,538*	7.5%*	169 s CPU*	≥3 questions / note	if text reader on
5	bev fallback for unusable answers	end	0 repairs	0	0%	0	only on failure	insurance
3	bev narrows candidates to 3	middle	17,284	0	0%	0	1 choice / call	drop
7	Route engineer queries	start	–	no data	–	–	1 choice / query	when built

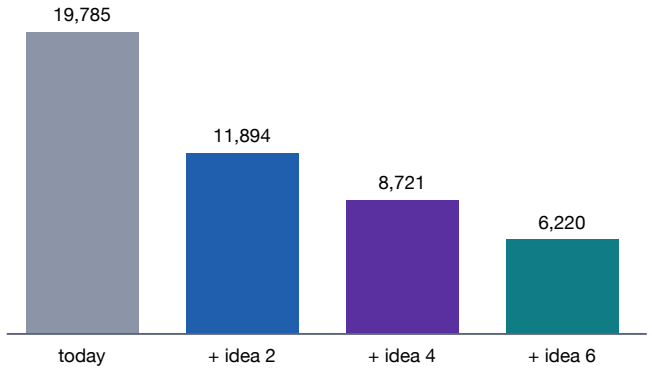
* v3 sends raw notes to the diagnostician, so its text reader made 0 calls. The text-reader figures come from the two "nudge" board runs (12 episodes) where it was on; there the total decode was 20,540 tokens. Ideas 2, 4 and 6 overlap (a skipped call has no citations to remove), so their sum is not the combined figure; see below.

Maximum decode removed, by idea



Decode tokens over the 12 v3 episodes (bar scale: 7,891 = 190 px).

If 2, 4 and 6 all worked (ceiling)



Total decode tokens, v3 board run. –68.6% at the end; the Llama part alone falls 64.0% (17,284 → 6,220).

Two ideas that look like savings but are not, today

Idea 3 (let bev narrow the cases so Llama answers less) looks like a decode saving. The board answers show it is not: all 509 Llama answers already rank **at most 3 cases** (mean 2.4), so a 3-case limit removes no tokens. It could still make the *prompt* shorter (fewer case lines to read), which is a prefill saving and was not estimated here. Idea 1 (replace the text reader) looks like the safest first step, but under v3 the text reader is not running, so it saves nothing in the configuration we benchmark today.

Where the decode goes today

agent (lane)	calls	decode / call	decode tokens	read tokens / call	decode share of call time	ms per decode token	ms per read token
Diagnostician, Llama 3.2 3B (NPU)	509	34.0	17,284	722	69.2%	64.3	1.27
Verifier, Gemma 3 1B (CPU)	100	25.0	2,501	400	35.2%	26.6	2.94
Text reader, Gemma 3 1B (CPU) · nudge runs	90	17.1	1,538	404	33.6%	37.0	2.98
for context: single agent, Llama 3.2 3B (NPU), 2 episodes	116	154.7	17,945	1,632	81.3%	69.7	1.49

The multi-agent already cut decode per diagnosis call **4.5×** against the single agent (154.7 → 34.0 tokens) by asking for short JSON made of line numbers. What is left is small per call but still dominant in time, because on the NPU a decoded token costs about 50× a read token. That is the opening for bev: it turns every answer into a read.

What the 34 tokens of a Llama answer are

A real answer from the board (A01, tick 33): `{"r":[[1,[1]],[2,[1]]],"sep":2,"n":[3,4],"x":[1]}`

part	meaning	tokens (all 509 answers)	who could produce it instead
`r` case lines	the ranked cases (≤3)	10,947 for everything except citations	stays with Llama (it is the diagnosis)
`sep, n, x`	separating case, notes used, unexplained facts		stays with Llama
fact lists inside `r`	the facts cited for each case	**5,756 (34.5% of answer text)**	bev, one choice per case (idea 4)
end token	the server's stop token	~1 per call	–

How bev's own cost compares

bev reads, it never writes. The decider already built on this branch sends the tick's fact lines plus the offered cases. For the 24 dev conformance requests that is a median of **347 tokens** to read (range 256–477; 3 to 6 options), counted with Qwen3's tokenizer, whose vocabulary is identical to bev's (151,643 entries, checked).

bev's time on the board has never been measured. Part C (bev running on the board, conformance with the reference, timing) has not been run. For scale only: 347 tokens read at the 3B model's measured NPU read rate would take ~0.44 s, against an average of 3.11 s per Llama call that idea 2 would skip. bev has about one tenth of the 3B model's matrix weights, but it is a different build (Q8_0, its own server), so this is a scale comparison, not an estimate of bev's speed.

One more cost: the board server **clears its memory between questions**, so three questions about the same facts read those facts three times. Ideas 1, 4 and 6 ask several questions per state. Keeping the state's cache across questions in `device/bev_decide/` would cut that repeat reading.

The ideas in detail

2 Call or skip the diagnostician start biggest saving, riskiest

What it does	Before a side's Llama call, code checks whether that side's *last* Llama answer already named the same top case as belief does now. Only then is bev asked to choose among the candidate cases. If bev also picks that case with high probability, the call is skipped and the side's last answer is re-used through the existing `side_answers` cache (the same path split-on-change already uses). Otherwise Llama is called as usual.
Why three must agree	A skip rule that trusts bev plus belief alone repeats the recorded decider problem: on the mock, a decider that backed belief undid Llama's corrections (library group top-1 0.736 → 0.659). Requiring Llama's own last answer to agree means a call is skipped only when Llama itself already said the same thing.
Measured ceiling	**236 of 509** Llama calls (46.4%) met the code part of the rule, carrying **7,891 decode tokens** and **733 s** of NPU call time. bev's agreement is unknown, so the real number is lower.
What skipping would have cost	In those 236 calls Llama's new top case equalled its previous one **201 times** (skipping changes nothing at rank 1). It changed 35 times: 8 changes moved into the true case's look-alike group (a skip would have lost them), 15 moved out of it (a skip would have avoided them), 12 left the group result the same.
Build	`agents/diagnostician.py` (the pre-check), `agents/decider.py` (a second question kind), `agents/gate.py` (re-use path, already present). The probability threshold is set on the dev set only.

Replay caveats: per-call top case, not the published ranking (the hybrid merge sits between); first-order only, a skipped call does not change later ticks in the replay; one run per episode.

4 bev picks the supporting facts middle second biggest

What it does	Llama names the cases only (`"r":[2,1]` instead of `"r":[[2,[1,3]],[1,[1]]]`). For each named case, bev answers "which fact line best supports this case?" as a choice over the tick's fact lines. Code writes the citation; the gate checks it as now (fact IDs carry their tick).
Measured ceiling	**5,756** of the 17,284 Llama decode tokens are citations: **33.3%**, or **369 s** of NPU decode.
bev's extra work	One choice per ranked case: about 1,222 questions over the run (2.4 cases × 509 calls).
Risk	bev returns one best fact; Llama today may cite several. The faithfulness score is built on citations, so it must be compared before and after. bev's "multi-hop reading and evidence" score is 70.6% in its model card (general text, not boiler facts).
Build	A compact prompt variant without fact lists, its grammar, the decider question, and expansion in the gate.

6 Replace the verifier replace measure first

What it does	For each claim, one yes/no: "Do the facts support this claim?" A probability threshold (set on dev) gives pass/fail. If any claim fails, one choice over fact lines gives the strongest contradiction.
Measured ceiling	All verifier decode: **2,501 tokens (12.6%)**, 100 calls, 189 s of CPU time. Claims per call: median 3, so about 300 yes/no questions.
Why "measure first"	Answer verification is bev's weakest area in its own card (54.1%). The verifier runs on the CPU lane in the background, so removing it does not speed up the diagnosis the demo shows. And an earlier Phase 2 finding was that the verifier never reorders a ranking, so its value has to be checked before swapping it.

1 Replace the text reader `replace` `only if the text reader is used`

What it does Each engineer note becomes a note-fact from bev questions instead of Gemma's generated JSON: *kind* = choice of OBS / MAINT / INSTR / OTHER; *subject* = choice from the fixed vocabulary; *state* = choice per subject. The gate checks the note-fact as now. Note reading is the kind of task bev scores best on (89.7% on support and intent routing in its card).

Measured ceiling In the nudge runs (text reader on): **90 notes, 1,538 decode tokens** (17.1 per note), 169 s of CPU call time; 7.5% of that configuration's decode. Under v3: **0**, because v3 sends raw notes to the diagnostician.

Snag The subject vocabulary (6 tags plus the asset model's equipment) is larger than the server's option limit (`--max-options 16` , a design value). It needs either a two-step question (tag or equipment, then which) or a raised limit.

5 bev fallback for unusable answers `end` `insurance`

What it does When a Llama answer cannot be used (wrong shape, cut off, failed call), ask bev to choose among the same offered cases instead of a second generating "repair" call.

Measured **0 repairs** in every board run checked (v3, both nudge runs, single agent): the grammar keeps the answers well-formed. So the saving is 0 today. It becomes useful if the model, the cap or the grammar changes.

3 bev narrows the candidates to 3 `middle` `drop as a decode idea`

Measured All 509 answers rank at most 3 cases already (median 2, mean 2.4), so cutting to 3 removes **0** decode tokens. Kept only as a possible prompt-length (prefill) idea, not estimated here.

7 Route engineer queries `start` `when the query agent is built`

What it does bev classifies a question into a fixed set of types; code answers the standard ones from templates; Llama is used only for open questions. The query agent's answer cap (120 tokens) is the largest in the plan.

Measured Nothing: the query agent has not been built, so there is no data.

Where bev must not go

- **Sensor/L1, triage, belief arithmetic, the gate, plant state.** Plain code: exact, instant, no decode to save. A model here would break invariant 1 (the model never sees raw numbers) or invariant 2 (plant state comes from facts).
- **Inside the tick.** bev's board time is unmeasured, so it cannot sit on the 200 ms hard path. It runs as a background job on its own lane (port 8082), like the other models.
- **Writing belief.** Like every model answer, bev's answers pass the gate and move log-odds by a capped amount, never directly.

Suggested order

step	what	why this order
0	Run Part C on the board (`HANDOFF_BEV_DECIDER_3.md`)	Nothing above is real until bev runs on the board, matches its reference, and has a measured time per question.
1	Idea 2, call or skip	Largest ceiling (up to 46% of Llama calls, 39.9% of all decode). Highest risk, so a conservative threshold and the existing pass rule decide it.
2	Idea 4, bev picks the evidence	Second ceiling (29.1%). Check faithfulness before and after.
3	Idea 6, verifier	12.6%, background lane only, weakest bev skill: measure it beside Gemma before replacing.
4	Ideas 1, 5, 7	Only when the text reader is on, the answers start failing, or the query agent exists.

How each idea would be judged (the project's existing rules)

- Each idea is a switch, off by default; with it off the run must be byte-identical to today (as was done for the decider).
- Each bev question is one fixed sentence, signed off by a human, never tuned. Every threshold is set on the dev set only; the 30 reporting episodes are run once, to report.
- Kept only if decode tokens fall **and** dev library group top-1 does not drop by more than the run-to-run spread (two baseline runs, as in the bev-decider-3 pass rule).
- Log for every call: read tokens, decode tokens, lane, time. bev calls log read tokens and time (decode is 0).

What these numbers are and are not

is	is not
Token counts and call times the board reported for the v3 run (8 Oct 2026, 12 episodes, 2,100 ticks, one run each, lockstep, agent code on the laptop, model calls on the QIDK). Text reader figures from the two nudge board runs.	A measurement of bev-decider. bev has never run on the board; its accuracy on boiler facts and its speed are unknown.
Ceilings: the decode removed if bev did its part perfectly.	A forecast of the saving, or of accuracy. Ideas 2 and 4 may lower accuracy; that is what the pass rule measures.
Call times on the board's NPU and CPU.	Energy. It was not measured, and is not estimated here.
Synthetic plant data from the project's generator.	Real plant results.

Method and checks

- Script: `bench/bev_savings_estimate.py` (reads `results/benchmarks/`, no model runs); tests: `tests/test_bev_savings_estimate.py` (4 tests).
- Answer surgery re-tokenises the real board answers with the Llama 3.2 3B tokenizer. Check: re-tokenised answers plus one end token per call give 17,212 tokens against the server's 17,284 (0.42% apart).
- Citation tokens counted two ways: rebuilding each answer's JSON without fact lists, and deleting the fact lists from the raw text with a pattern. Both give 5,756.
- Idea 2's replay needs each call's side. Sides are paired by prompt size within a tick and checked against the sides of the cases each call was offered: 506 consistent, 0 inconsistent, 3 with no case to check.
- Mutation check: reversing the skip condition (`p == lead` → `p != lead`) fails `test_replay_counts_only_calls_where_last_answer_equals_belief_leader`; restored, all 4 pass.
- bev's read tokens: the 24 dev conformance requests in `reports/data/bev_requests.json`, counted with the Qwen3 tokenizer (vocabulary identical to bev's).